

Specification-based testing



This chapter covers

- Creating test cases using specification-based testing
- Identifying and creating test cases for program boundaries

Software requirements are undoubtedly the most valuable artifact of software testing. By *requirements*, I mean any textual document that describes what a functionality should do. Requirements tell us precisely what the software needs to do and what it should not do. They describe the intricacies of the business rules that the software has to implement and we need to validate. Therefore, requirements should be the first artifact you go for when it comes to testing!

In this chapter, we explore *specification-based testing*. These techniques use the program requirements—such as agile user stories or UML use cases—as testing input. We will discuss how to use all the information available in a requirement to systematically derive a set of tests that exercise that requirement extensively.

Where does specification-based testing fit into the entire testing process? Imagine that a software developer receives a new feature to implement. The developer writes the implementation code, guided by test-driven development (TDD)

cycles, and always ensures that the code is testable. With all the classes ready, the developer switches to “testing mode.” It is time to systematically look for bugs. This is where specification testing fits in: it is the first testing technique I recommend using once you’re in testing mode.

As I mentioned, the idea of specification-based testing is to derive tests from the requirements themselves. The specific implementation is less important. Of course, we use source code to test, too—this *structural testing* is the next technique in the workflow. Once you have a complete picture of all the techniques, you will be able to use them iteratively and go back and forth between them.

2.1 The requirements say it all

Let’s start with an example. A new set of requirements comes in for you to develop. As soon as you begin to analyze the requirements, you identify a particular method you need to implement: a method that searches for substrings between two tags in a given string and returns all the matching substrings. Let’s call this method `substringsBetween()`, inspired by the Apache Commons Lang library (<http://mng.bz/nYR5>). You are about to test a real-world open source method.

After some thinking, you end up with the following requirements for the `substringsBetween()` method:

Method: `substringsBetween()`

Searches a string for substrings delimited by a start and end tag, returning all matching substrings in an array.

- `str`—The string containing the substrings. Null returns null; an empty string returns another empty string.
- `open`—The string identifying the start of the substring. An empty string returns null.
- `close`—The string identifying the end of the substring. An empty string returns null.

The program returns a string array of substrings, or null if there is no match.

Example: if `str` = “`axcaycazc`”, `open` = “`a`”, and `close` = “`c`”, the output will be an array containing [“`x`”, “`y`”, “`z`”]. This is the case because the “`a<something>c`” substring appears three times in the original string: the first contains “`x`” in the middle, the second “`y`,” and the last “`z`.”

With these requirements in mind, you write the implementation shown in listing 2.1. You may or may not use TDD (discussed in chapter 8) to help you develop this feature. You are somewhat confident that the program works. Slightly, but not completely.

Listing 2.1 Implementing the `substringsBetween()` method

```

public static String[] substringsBetween(final String str,
    final String open, final String close) {

    if (str == null || isEmpty(open) || isEmpty(close)) {
        return null;
    }

    int strLen = str.length();
    if (strLen == 0) {
        return EMPTY_STRING_ARRAY;
    }

    int closeLen = close.length();
    int openLen = open.length();
    List<String> list = new ArrayList<>();
    int pos = 0;

    while (pos < strLen - closeLen) {
        int start = str.indexOf(open, pos);

        if (start < 0) {
            break;
        }

        start += openLen;
        int end = str.indexOf(close, start);
        if (end < 0) {
            break;
        }

        list.add(str.substring(start, end));
        pos = end + closeLen;

        if (list.isEmpty()) {
            return null;
        }

        return list.toArray(EMPTY_STRING_ARRAY);
    }
}

```

If the pre-conditions do not hold, returns null right away

If the string is empty, returns an empty array immediately

A pointer that indicates the position of the string we are looking at

Looks for the next occurrence of the open tag

Breaks the loop if the open tag does not appear again in the string

Looks for the close tag

Breaks the loop if the close tag does not appear again in the string

Returns null if we do not find any substrings

Gets the substring between the open and close tags

Moves the pointer to after the close tag we just found

Let's walk through an example. Consider the inputs `str = "axcaycazc"`, `open = "a"`, and `close = "c"`. None of the three strings are empty, so the method goes straight to the `openLen` and `closeLen` variables. These two variables store the length of the open and close strings, respectively. In this case, both are equal to 1, as "a" and "c" are strings with a single character.

The program then goes into its main loop. This loop runs while there still may be substrings in the string to check. In the first iteration, `pos` equals zero (the beginning of the string). We call `indexOf`, looking for a possible occurrence of the open tag. We

pass the open tag and the position to start the search, which at this point is 0. `indexOf` returns 0, which means we found an open tag. (The first element of the string is already the open tag.)

The program then looks for the end of the substring by calling the `indexOf` method again, this time on the `close` tag. Note that we increase the start position by the length of the open tag because we want to look for the `close` tag after the end of the entire open tag. Remember that the open tag has a length of one but can have any length. If we find a `close` tag, this means there is a substring to return to the user. We get this substring by calling the `substring` method with the start and end positions as parameters. We then reposition our `pos` pointer, and the loop iterates again. Figure 2.1 shows the three iterations of the loop as well as the locations to which the main pointers (`start`, `end`, and `pos`) are pointing.

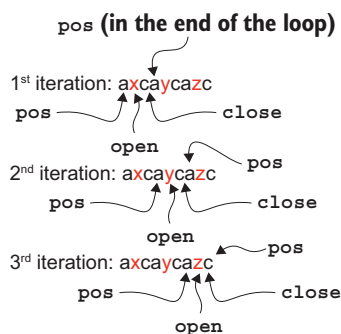


Figure 2.1 The three iterations of the `substringsBetween` method for our example

Now that you have finished the first implementation, you flip your mind to testing mode. It is time for specification and boundary testing. As an exercise, before we work on this problem together, look at the requirements one more time and write down all the test cases you can come up with. The format does not matter—it can be something like “all parameters null.” When you are finished with this chapter, compare your initial test suite with the one we are about to derive together.

The best way to ensure that this method works properly would be to test all the possible combinations of inputs and outputs. Given that `substringsBetween()` receives three string parameters as an input, we would need to pass all possible valid strings to the three parameters, combined in all imaginable ways. As we discussed in chapter 1, exhaustive testing is rarely possible. We have to be pragmatic.

2.1.1 Step 1: Understanding the requirements, inputs, and outputs

Regardless of how your requirements are written (or even if they are only in your mind), they include three parts. First is what the program/method must do: its business rules. Second, the program receives data as inputs. Inputs are a fundamental part of our reasoning, as it is through them that we can test the different cases. Third, reasoning about the output will help us better understand what the program does and how the inputs are converted to the expected output.

For the `substringsBetween()` method, my reasoning would be

- 1 The *goal* of this method is to collect all substrings in a string that are delimited by an open tag and a close tag (the user provides these).
- 2 The program receives three *parameters*:
 - a `str`, which represents the string from which the program will extract substrings
 - b The open tag, which indicates the start of a substring
 - c The close tag, which indicates the end of the substring
- 3 The program *returns* an array composed of all the substrings found by the program.

Such reflection is useful to think about what you want from the method.

2.1.2 Step 2: Explore what the program does for various inputs

An ad hoc exploration of what the method does may increase your understanding of it. I have noticed this when observing professional software developers writing test cases for methods they have never seen before (Aniche, Treude, and Zaidman, 2021). This step is more relevant when you did not write the code—if you wrote it, this exploration phase may not be needed.

To illustrate this step, suppose you did not write this code (which, in this case, is true). My process would be as follows (see the JUnit code in listing 2.2):

Let's see the program working on a happy case. I will pass the string "abcd" with the open tag "a" and the close tag "d". I expect it to return an array with a single element: ["bc"]. I try that (in a unit test), and the program returns what I expect.

Next, let's see what happens if there are multiple substrings in the main string. I will pass the string "abcd**ab**cdab" with the same open and close tags. I expect it to return an array with two strings: ["bc", "bc"]. The program returns what I expect.

I expect the program to behave the same with open and close tags larger than a single character. I will repeat the second test, doubling the "a"s and the "d"s in all the parameters. I will also change one of the "bc"s to "bf", so it is easier to check that the method returns two different substrings: ["bc", "bf"]. The program returns what I expect.

Listing 2.2 Exploratory tests for `substringsBetween()`

```
@Test
void simpleCase() {
    assertThat(
        StringUtils.substringsBetween("abcd", "a", "d")
    ).isEqualTo(new String[] { "bc" });
}
```

← We write these test cases based on our feelings.
What do we want to explore next?

```
@Test
void manySubstrings() {
    assertThat(
```

← I don't care if they are good tests, as long as they teach me something about the code.

```

        StringUtils.substringsBetween("abcdabcdab", "a", "d")
    ).isEqualTo(new String[] { "bc", "bc" });
}

@Test
void openAndCloseTagsThatAreLongerThan1Char() {
    assertThat(
        StringUtils.substringsBetween("aabcdadaabfddaab", "aa", "dd")
    ).isEqualTo(new String[] { "bc", "bf" });
}

```

I wrote all the test code in a single line, although you cannot see that in the printed book. Feel free to write it any way you prefer.

I stop this exploration phase when I have a clear mental model of how the program should work. Note that I do not expect you to perform the same exploration I did—it is personal and guided by my hypothesis about the program. Also note that I did not explore any corner cases; that comes later. At this moment, I am only interested in better understanding the program.

2.1.3 Step 3: Explore possible inputs and outputs, and identify partitions

We should find a way to prioritize and select a subset of inputs and outputs that will give us sufficient certainty about the correctness of the program. Although the number of possible program inputs and outputs is nearly infinite, some sets of inputs make the program behave the same way, regardless of the precise input value.

In the case of our example, for testing purposes, the input “abcd” with open tag “a” and close tag “d”, which makes the program return “bc”, is the same as the input “xyzw” with open tag “x” and close tag “w”. You change the letters, but you expect the program to do the same thing for both inputs. Given your resource constraints, you will test just one of these inputs (it does not matter which), and you will trust that this single case represents that entire class of inputs. In testing terminology, we say that these two inputs are *equivalent*.

Once you have identified this class (or partition), you repeat the process and look for another class that will make the program behave in a different way that you have not yet tested. If you keep dividing the domain, you will eventually identify all the different possible classes (or partitions) of inputs.

A systematic way to do such an exploration is to think of the following:

- 1 Each input individually: “What are the possible classes of inputs I can provide?”
- 2 Each input in combination with other inputs: “What combinations can I try between the open and close tags?”
- 3 The different classes of output expected from this program: “Does it return arrays? Can it return an empty array? Can it return nulls?”

I find it easiest to start with individual inputs. Follow me:

- *str parameter*—The string can be any string. The specification mentions the null and empty cases; I would have tested those anyway, because they are always

good exceptional test cases. Given that this is a string (which is basically a list of characters), I will also test what happens if the string has length 1.

- a Null string
- b Empty string
- c String of length 1
- d String of length > 1 (any string)
- *open parameter*—This can also be anything. I will try it with null and empty, as I learned from the *str* parameter that those cases are special in this program. I will also try strings with length 1 and greater than 1:
 - a Null string
 - b Empty string
 - c String of length 1
 - d String of length > 1
- *close parameter*—This parameter is like the previous one:
 - a Null string
 - b Empty string
 - c String of length 1
 - d String of length > 1

Once the input variables are analyzed in detail, we explore possible *combinations* of variables. A program's input variables may be related to each other. In the example, it is clear that the three variables have a dependency relationship. Follow me again:

- *(str, open, close) parameters*—open and close may or may not be in the string. Also, open may be there, but not close (and vice versa).
 - a *str* contains neither the open nor the close tag.
 - b *str* contains the open tag but not the close tag.
 - c *str* contains the close tag but not the open tag.
 - d *str* contains both the open and close tags.
 - e *str* contains both the open and close tags multiple times.

Note that this thought process depended on my experience as a tester. The documentation does not explicitly mention tags not being in the string, nor does it mention the open tag being present but the close tag not. I saw this case because of my experience as a tester.

Finally, we reflect on the possible outputs. The method returns an array of sub-strings. I can see a set of possible different outputs, both for the array itself and for the strings within the array:

- *Array of strings* (output)
 - a Null array
 - b Empty array

- c Single item
 - d Multiple items
- *Each individual string* (output)
 - a Empty
 - b Single character
 - c Multiple characters

You may think that reflecting on the outputs is not necessary. After all, if you reasoned correctly about the inputs, you are probably exercising all the possible kinds of outputs. This is a valid argument. Nevertheless, for more complex programs, reflecting on the outputs may help you see an input case that you did not identify before.

2.1.4 Step 4: Analyze the boundaries

Bugs in the boundaries of the input domain are common in software systems. As developers, we have all made mistakes such as using a “greater than” operator ($>$) where it should have been a “greater than or equal to” operator ($>=$). Programs with such bugs tend to work well for most provided inputs, but they fail when the input is near the boundary. Boundaries are everywhere, and our goal in this section is to learn how to identify them.

When we devise partitions, they have *close boundaries* with the other partitions. Imagine a simple program that prints “hiphip” if the given input is a number smaller than 10 or “hooray” if the given input is greater than or equal to 10. A tester can divide the input domain into two partitions: (1) the set of inputs that make the program print “hiphip” and (2) the set of inputs that make the program print “hooray”. Figure 2.2 illustrates this program’s inputs and partitions. Note that the input value 9 belongs to the “hiphip” partition, while the input value 10 belongs to the “hooray” partition.

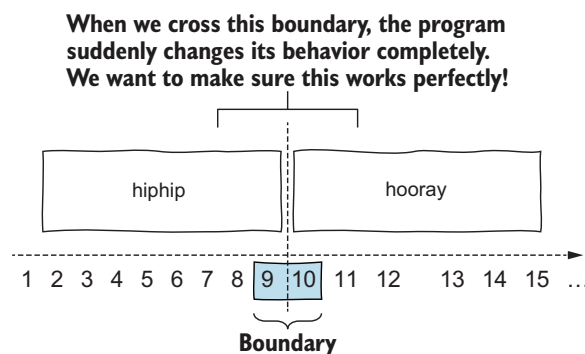


Figure 2.2 The boundary between the “hiphip” and “hooray” partitions. Numbers up to 9 belong to the “hiphip” partition, and numbers greater than 9 belong to the “hooray” partition.

The odds of a programmer writing a bug near the boundary (in this case, near the input values 9 and 10) are greater than for other input values. This is what boundary testing is about: making the program behave correctly when inputs are near a boundary. And this is what this fourth step is about: boundary testing.

Whenever a boundary is identified, I suggest that you test what happens to the program when inputs go from one boundary to the other. In the previous example, this would mean having a test with 9 as input and another test with 10 as input. This idea is similar to what Jeng and Weyuker proposed in their 1994 paper: testing two points whenever there is a boundary. One test is for the *on point*, which is the point that is *on* the boundary; and the other test is for the *off point*, which is the point closest to the boundary that belongs to the partition the on point does not belong to (that is, the other partition).

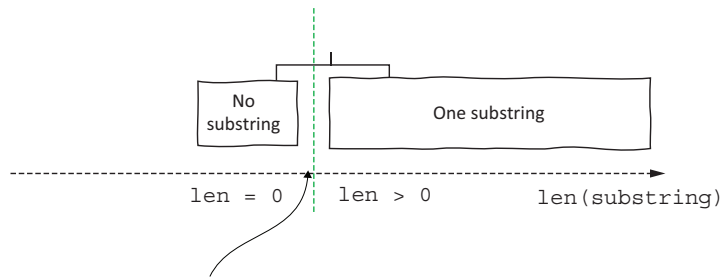
In the *hiphip-hooray* example, the on point is 10. Note that 10 is the number that appears in the specification of the program (`input >= 10`) and is likely to also be the number the developer uses in the `if` statement. The value 10 makes the program print “hooray”. The off point is the point closest to the boundary that belongs to the other partition. In this case, the off point is 9. The number 9 is the closest number to 10, and it belongs to the “hiphip” partition.

Let’s discuss two more common terms: *in point* and *out point*. In points are points that make the condition true. You may have an infinite number of them. In the *hiphip-hooray* example, 11, 12, 25, and 42 are all examples of in points. Out points, on the other hand, are points that make the condition false. 8, 7, 2, and -42 are all examples of out points. In equalities, the in point is the one in the condition, and all others are out points. For example, in `a == 10`, 10 is the (only) in point and the on point; 12 is an out point and an off point; and 56 is an out point. Whenever you find a boundary, two tests (for the on and off points) are usually enough, although, as I will discuss later, I do not mind throwing in some interesting in and out points to have a more complete test suite.

Another common situation in boundary testing is finding boundaries that deal with equalities. In the previous example, suppose that instead of `input >= 10`, the specification says that the program prints “hooray” whenever the input is 10 or “hiphip” otherwise. Given that this is an equality, we now have one on point (10) but two off points (9 and 11), because the boundary applies to both sides. In this case, as a tester, you would write three test cases.

My trick to explore boundaries is to look at all the partitions and think of inputs between them. Whenever you find one that is worth testing, you test it.

In our example, a straightforward boundary happens when the string passes from empty to non-empty, as you know that the program stops returning empty and will (possibly) start to return something. You already covered this boundary, as you have partitions for both cases. As you examine each partition and how it makes boundaries with others, you analyze the partitions in the (`str`, `open`, `close`) category. The program can have no substrings, one substring, or multiple substrings. And the `open` and `close` tags may not be in the string; or, more importantly, they may be in the string, but with no substring between them. This is a boundary you should exercise! See figure 2.3.



When the input contains both the “open” and “close” tags, and the length of the substring changes from 0 to greater than 0, the program starts to return this substring. It’s a boundary, and we should exercise it!

Figure 2.3 Some of the boundaries in the `substringsBetween()` problem.

Whenever we identify a boundary, we devise two tests for it, one for each side of the boundary. For the “no substring”/“one substring” boundary, the two tests are as follows:

- `str` contains both open and close tags, with *no* characters between them.
- `str` contains both open and close tags, with characters between them.

The second test is not necessary in this case, as other tests already exercise this situation. Therefore, we can discard it.

2.1.5 Step 5: Devise test cases

With the inputs, outputs, and boundaries properly dissected, we can generate concrete test cases. Ideally, we would combine all the partitions we’ve devised for each of the inputs. The example has four categories, each with four or five partitions: the `str` category with four partitions (*null string*, *empty string*, *string of length 1*, and *string of length > 1*), the open category with four partitions (the same as `str`), the close category with four partitions (also the same as `str`), and the (`str`, open, close) category with five partitions (*string does not contain either the open or close tags*, *string contains the open tag but does not contain the close tag*, *string contains the close tag but does not contain the open tag*, *string contains both the open and close tags*, *string contains both the open and close tags multiple times*). This means you would start with the `str` null partition and combine it with the partitions of the open, close, and (`str`, open, close) categories. You would end up with $4 \times 4 \times 4 \times 5 = 320$ tests. Writing 320 tests may be an effort that will not pay off.

In such situations, we pragmatically decide which partitions should be combined with others and which should not. A first idea to reduce the number of tests is to test exceptional cases only once and not combine them. For example, the *null string* partition may be tested only once and not more than that. What would we gain from combining *null string* with open being null, empty, length = 1, and length > 1 as well as with close being null, empty, length = 1, length > 1, and so on? It would not be worth the effort. The same goes for *empty string*: one test may be good enough. If we

apply the same logic to the other two parameters and test them as null and empty just once, we already drastically reduce the number of test cases.

There may be other partitions that do not need to be combined fully. In this problem, I see two:

- For the *string of length 1* case, given that the string has length 1, two tests may be enough: one where the single character in the string matches open and close, and one where it does not.
- Unless we have a good reason to believe that the program handles open and close tags of different lengths in different ways, we do not need the four combinations of (*open length = 1, close length = 1*), (*open length > 1, close length = 1*), (*open length = 1, close length > 1*), and (*open length > 1, close length > 1*). Just (*open length = 1, close length = 1*) and (*open length > 1, close length > 1*) are enough.

In other words, do not blindly combine partitions, as doing so may lead to less relevant test cases. Looking at the implementation can also help you reduce the number of combinations. We discuss using the source code to design test cases in chapter 3.

In the following list, I've marked with an [x] partitions we will not test multiple times:

- str—Null string [x], empty string [x], length = 1 [x], length > 1
- open—Null string [x], empty string [x], length = 1, length > 1
- close—Null string [x], empty string [x], length = 1, length > 1
- str—Null string [x], empty string [x], length = 1, length > 1
- (str, open, close)—String does not contain either the open or the close tag, string contains the open tag but does not contain the close tag, string contains the close tag but does not contain the open tag, string contains both the open and close tags, string contains both the open and close tags multiple times

With a clear understanding of which partitions need to be extensively tested and which ones do not, we can derive the test cases by performing the combination. First, the exceptional cases:

- T1: str is null.
- T2: str is empty.
- T3: open is null.
- T4: open is empty.
- T5: close is null.
- T6: close is empty.

Then, *str length = 1*:

- T7: The single character in str matches the open tag.
- T8: The single character in str matches the close tag.
- T9: The single character in str does not match either the open or the close tag.
- T10: The single character in str matches both the open and close tags.

Now, $str\ length > 1$, $open\ length = 1$, $close = 1$:

- T11: str does not contain either the open or the close tag.
- T12: str contains the open tag but does not contain the close tag.
- T13: str contains the close tag but does not contain the open tag.
- T14: str contains both the open and close tags.
- T15: str contains both the open and close tags multiple times.

Next, $str\ length > 1$, $open\ length > 1$, $close > 1$:

- T16: str does not contain either the open or the close tag.
- T17: str contains the open tag but does not contain the close tag.
- T18: str contains the close tag but does not contain the open tag.
- T19: str contains both the open and close tags.
- T20: str contains both the open and close tags multiple times.

Finally, here is the test for the boundary:

- T21: str contains both the open and close tags with no characters between them.

We end up with 21 tests. Note that deriving them did not require much creativity: the process we followed was systematic. This is the idea!

2.1.6 Step 6: Automate the test cases

It is now time to transform the test cases into automated JUnit tests. Writing those tests is mostly a mechanical task. The creative part is coming up with inputs to exercise the specific partition and understanding the correct program output for that partition.

The automated test suite is shown in listings 2.3 through 2.7. They are long but easy to understand. Each call to the `substringsBetween` method is one of our test cases. The 21 calls to it are spread over the test methods, each matching the test cases we devised earlier.

First are the tests related to the string being null or empty.

Listing 2.3 Tests for `substringsBetween`, part 1

```
import org.junit.jupiter.api.Test;
import static ch2.StringUtils.substringsBetween;
import static org.assertj.core.api.Assertions.assertThat;

public class StringUtilsTest {

    @Test
    void strIsNullOrEmpty() {
        assertThat(substringsBetween(null, "a", "b"))
            .isEqualTo(null);
```

This first call to
`substringsBetween`
is our test T1.

```

    assertThat(substringsBetween("", "a", "b"))
        .isEqualTo(new String[] {});
}
}

```

← Test T2

Next are all the tests related to open or close being null or empty.

Listing 2.4 Tests for substringsBetween, part 2

```

@Test
void openIsNullOrEmpty() {
    assertThat(substringsBetween("abc", null, "b")).isEqualTo(null);
    assertThat(substringsBetween("abc", "", "b")).isEqualTo(null);
}

@Test
void closeIsNullOrEmpty() {
    assertThat(substringsBetween("abc", "a", null)).isEqualTo(null);
    assertThat(substringsBetween("abc", "a", "")).isEqualTo(null);
}

```

Now come all the tests related to string and open and close tags with length 1.

Listing 2.5 Tests for substringsBetween, part 3

```

@Test
void strOfLength1() {
    assertThat(substringsBetween("a", "a", "b")).isEqualTo(null);
    assertThat(substringsBetween("a", "b", "a")).isEqualTo(null);
    assertThat(substringsBetween("a", "b", "b")).isEqualTo(null);
    assertThat(substringsBetween("a", "a", "a")).isEqualTo(null);
}

@Test
void openAndCloseOfLength1() {
    assertThat(substringsBetween("abc", "x", "y")).isEqualTo(null);
    assertThat(substringsBetween("abc", "a", "y")).isEqualTo(null);
    assertThat(substringsBetween("abc", "x", "c")).isEqualTo(null);
    assertThat(substringsBetween("abc", "a", "c"))
        .isEqualTo(new String[] {"b"});
    assertThat(substringsBetween("abcabc", "a", "c"))
        .isEqualTo(new String[] {"b", "b"});
}

```

Then we have the tests for the open and close tags of varying sizes.

Listing 2.6 Tests for substringsBetween, part 4

```

@Test
void openAndCloseTagsOfDifferentSizes() {
    assertThat(substringsBetween("aabcc", "xx", "yy")).isEqualTo(null);
    assertThat(substringsBetween("aabcc", "aa", "yy")).isEqualTo(null);
}

```

```

    assertThat(substringsBetween("aabcc", "xx", "cc")).isEqualTo(null);
    assertThat(substringsBetween("aabbcc", "aa", "cc"))
        .isEqualTo(new String[] { "bb" });
    assertThat(substringsBetween("aabbccaaeecc", "aa", "cc"))
        .isEqualTo(new String[] { "bb", "ee" });
}

```

Finally, here is the test for when there is no substring between the open and close tags.

Listing 2.7 Tests for `substringsBetween`, part 5

```

@Test
void noSubstringBetweenOpenAndCloseTags() {
    assertThat(substringsBetween("aabb", "aa", "bb"))
        .isEqualTo(new String[] { "" });
}

```

I decided to group the assertions in five different methods. They almost match my groups when engineering the test cases in step 5. The only difference is that I broke the exceptional cases into three test methods: `strIsNullOrEmpty`, `openIsNullOrEmpty`, and `closeIsNullOrEmpty`.

Some developers would vouch for a single method per test case, which would mean 21 test methods, each containing one method call and one assertion. The advantage would be that the test method's name would clearly describe the test case. JUnit also offers the `ParameterizedTest` feature (<http://mng.bz/voKp>), which could be used in this case.

I prefer simple test methods that focus on one test case, especially when implementing complex business rules in enterprise systems. But in this case, there are lots of inputs to test, and many of them are variants of a larger partition, so it made more sense to me to code the way I did.

Deciding whether to put all tests in a single method or in multiple methods is highly subjective. We discuss test code quality and how to write tests that are easy to understand and debug in chapter 10.

Also note that sometimes there are values we do not care about. For example, consider test case 1: `str` is null. We do not care about the values we pass to the open and close tags here. My usual approach is to select reasonable values for the inputs I do not care about—that is, values that will not interfere with the test.

2.1.7 Step 7: Augment the test suite with creativity and experience

Being systematic is good, but we should never discard our experience. In this step, we look at the partitions we've devised and see if we can develop interesting variations. Variation is always a good thing to have in testing.

In the example, when revisiting the tests, I noticed that we never tried strings with spaces. I decided to engineer two extra tests based on T15 and T20, both about “`str` contains both open and close tags multiple times”: one for open and close tags with

lengths 1, another for open and close tags with larger lengths. These check whether the implementation works if there are whitespaces in the string. You see them in listing 2.8.

NOTE It's possible we don't need to test for this extra case. Maybe the implementation handles strings in a generic way. For now, we are only looking at the requirements, and testing special characters is always a good idea. If you have access to the implementation (as we discuss in the next chapter), the code can help you decide whether a test is relevant.

Listing 2.8 Tests for `substringsBetween` using parameterized tests, part 6

```
@Test
void openAndCloseOfLength1() {
    // ... previous assertions here
    assertThat(substringsBetween("abcabyt byrc", "a", "c"))
        .isEqualTo(new String[] {"b", "byt byr"});
}

@Test
void openAndCloseTagsOfDifferentSizes() {
    // ... previous assertions here
    assertThat(substringsBetween("a abb ddc ca abbcc", "a a", "c c"))
        .isEqualTo(new String[] {"bb dd"});
}
```

We end up with 23 test cases. Take time to revisit all the steps we have worked through, and then consider this question: are we finished?

We are finished with specification testing. However, we are not done testing. After specification testing, the next step is to bring the implementation into play and augment our test suite with what we see in the code. That is the topic of chapter 3.

Four eyes are better than two

A reviewer of this book had an interesting question: what about a test case where the input is **aabccddaabeddaab**, open is **aa**, and close is **d**? “bc” and “be” are the substrings between the provided open and the close tags (aa<bcb>ddaa<bbe>ddaab), but “bcddaabed” could also be considered a substring (aa<bccddaabed>daab).

At first, I thought I had missed this test case. But in fact, it is the same as T15 and T20.

Different people approach problems in different ways. My thought process was, “Let’s see if the program breaks if we have multiple open and close tags in the string.” The reviewer may have thought, “Let’s see if the program will incorrectly go for the longer substring.”

We want to make testing as systematic as possible, but a lot depends on how the developer models the problem. Sometimes you will not see all the test cases. When you do come up with a new test, add it to the test suite!

2.2 Specification-based testing in a nutshell

I propose a seven-step approach to derive systematic tests based on a specification. This approach is a mix of the *category-partition method* proposed by Ostrand and Balcer in their seminal 1988 work, and Kaner et al.'s *Domain Testing Workbook* (2013), with my own twist: see figure 2.4.

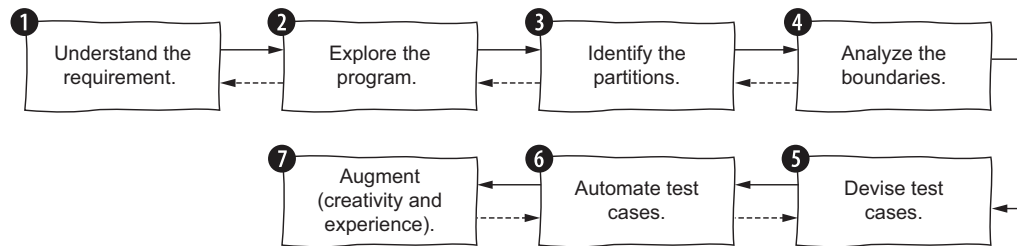


Figure 2.4 The seven steps I propose to derive test cases based on specifications. The solid arrows indicate the standard path to follow. The dashed arrows indicate that, as always, the process should be iterative, so in practice you'll go back and forth until you are confident about the test suite you've created.

The steps are as follows:

- 1 *Understand the requirement, inputs, and outputs.* We need an overall idea of what we are about to test. Read the requirements carefully. What should the program do? What should it not do? Does it handle specific corner cases? Identify the input and output variables in play, their types (integers, strings, and so on), and their input domain (for example, is the variable a number that must be between 5 and 10?). Some of these characteristics can be found in the program's specification; others may not be stated explicitly. Try to understand the nitty-gritty details of the requirements.
- 2 *Explore the program.* If you did not write the program yourself, a very good way to determine what it does (besides reading the documentation) is to play with it. Call the program under test with different inputs and see what it produces as output. Continue until you are sure your mental model matches what the program does. This exploration does not have to be (and should not be) systematic. Rather, focus on increasing your understanding. Remember that you are still not testing the program.
- 3 *Judiciously explore the possible inputs and outputs, and identify the partitions.* Identifying the correct partitions is the hardest part of testing. If you miss one, you may let a bug slip through. I propose three steps to identify the partitions:
 - a Look at each input variable individually. Explore its type (is it an integer? is it a string?) and the range of values it can receive (can it be null? is it a number ranging from 0 to 100? does it allow negative numbers?).
 - b Look at how each variable may interact with another. Variables often have dependencies or put constraints on each other, and those should be tested.

- c Explore the possible types of outputs, and make sure you are testing them all. While exploring the inputs and outputs, pay attention to any implicit (business) rules, logic, or expected behavior.
- 4 *Identify the boundaries.* Bugs love boundaries, so be extra thorough here. Analyze the boundaries of all the partitions you devised in the previous step. Identify the relevant ones, and add them to the list.
- 5 *Devise test cases based on the partitions and boundaries.* The basic idea is to combine all the partitions in the different categories to test all possible combinations of inputs. However, combining them all may be too expensive, so part of the task is to reduce the number of combinations. The common strategy is to test exceptional behavior only once and not combine it with the other partitions.
- 6 *Automate the test cases.* A test is only a test when it is automated. Therefore, the goal is to write (JUnit) automated tests for all the test cases you just devised. This means identifying concrete input values for them and having a clear expectation of what the program should do (the output). Remember that test code is code, so reduce duplication and ensure that the code is easy to read and that the different test cases are easily identifiable in case one fails.
- 7 *Augment the test suite with creativity and experience.* Perform some final checks. Revisit all the tests you created, using your experience and creativity. Did you miss something? Does your gut feeling tell you that the program may fail in a specific case? If so, add a new test case.

2.3 *Finding bugs with specification testing*

The developers of the Apache Commons Lang framework (the framework where I extracted the implementation of the `substringsBetween` method) are just too good. We did not find any bugs there. Let's look at another example: one implemented by me, an average developer who makes mistakes from time to time. This example will show you the value of specification testing. Try to spot the bug before I reveal it!

Some friends and I have participated in many coding challenges, primarily for fun. A couple of years ago we worked on the following problem inspired by LeetCode (<https://leetcode.com/problems/add-two-numbers/>):

The method receives two numbers, `left` and `right` (each represented as a list of digits), adds them, and returns the result as a list of digits.

Each element in the `left` and `right` lists of digits should be a number from [0–9]. An `IllegalArgumentException` is thrown if this pre-condition does not hold.

- `left`—A list containing the left number. Null returns null; empty means 0.
- `right`—A list containing the right number. Null returns null; empty means 0.

The program returns the sum of `left` and `right` as a list of digits.

For example, adding the numbers 23 and 42 means a (left) list with two elements [2,3], a (right) list with two elements [4,2] and, as an output, a list with two elements [6,5] (since $23 + 42 = 65$).

My initial implementation was as follows.

Listing 2.9 Initial implementation of the add() method

```
public List<Integer> add(List<Integer> left, List<Integer> right) {
    if (left == null || right == null)
        return null;

    Collections.reverse(left);
    Collections.reverse(right);

    LinkedList<Integer> result = new LinkedList<>();

    int carry = 0;

    for (int i = 0; i < max(left.size(), right.size()); i++) {

        int leftDigit = left.size() > i ? left.get(i) : 0;
        int rightDigit = right.size() > i ? right.get(i) : 0;

        if (leftDigit < 0 || leftDigit > 9 ||
            rightDigit < 0 || rightDigit > 9)
            throw new IllegalArgumentException();

        int sum = leftDigit + rightDigit + carry;

        result.addFirst(sum % 10);

        carry = sum / 10;
    }

    return result;
}
```

← Returns null if left or right is null

← Reverses the numbers so the least significant digit is on the left

← While there is a digit, keeps summing, taking carries into consideration

← Throws an exception if the pre-condition does not hold

← Sums the left digit with the right digit with the possible carry

← The digit should be a number between 0 and 9. We calculate it by taking the rest of the division (the % operator) of the sum by 10.

← If the sum is greater than 10, carries the rest of the division to the next digit

The algorithm works as follows. First it reverses both lists of digits, so the least significant digit is on the left. This makes it easier for us to loop through the list. Then, for each digit in both the left and right numbers, the algorithm gets the next relevant digits and sums them. If the resulting sum is greater than 10, +1 needs to be carried to the next most significant digit. In the end, the algorithm returns the list.

I was just having fun with coding, so I did not write systematic tests. I tried a couple of inputs and observed that the output was correct. If you already understand the concept of code coverage, these four tests achieve 100% branch coverage if we discard the ifs related to checking null and pre-conditions (if you are not familiar with code coverage, don't worry; we discuss it in the next chapter):

- $T1 = [1] + [1] = [2]$
- $T2 = [1,5] + [1,0] = [2,5]$

- $T3 = [1,5] + [1,5] = [3,0]$
- $T4 = [5,0,0] + [2,5,0] = [7,5,0]$

The program worked fine for these inputs. I submitted it to the coding challenge platform, and, to my surprise, the implementation was rejected! There was a bug in my code. Before I show you where it is, here is how specification testing would have caught it.

First we analyze each parameter in isolation:

- *left parameter*—It is a list, so we should first exercise basic inputs such as null, empty, a single digit, and multiple digits. Given that this list represents a number, we should also try a number with many zeroes on the left. Such zeroes are useless, but it is good to see whether the implementation can handle them. Thus we have the following partitions:
 - Empty
 - Null
 - Single digit
 - Multiple digits
 - Zeroes on the left
- *right parameter*—We have the same list of partitions as for the left parameter:
 - Empty
 - Null
 - Single digit
 - Multiple digits
 - Zeroes on the left

left and *right* have a relationship. Let's explore that:

- (*left*, *right*) *parameters*—They can be different sizes, and the program should be able to handle it:
 - $\text{length}(\text{left list}) > \text{length}(\text{right list})$
 - $\text{length}(\text{left list}) < \text{length}(\text{right list})$
 - $\text{length}(\text{left list}) = \text{length}(\text{right list})$

While not explicit in the documentation, we know that the sum of two numbers should be the same regardless of whether the highest number is on the left or right side of the equation. We also know that some sums require carrying. For example, suppose we're summing $18 + 15$: $8 + 5 = 13$, which means we have a 3, and we carry +1 to the next digit. We then add $1 + 1 + 1$: the first 1 from the left number, the second 1 from the right number, and the third 1 carried from the previous sum. The final result is 33. Figure 2.5 illustrates this process.

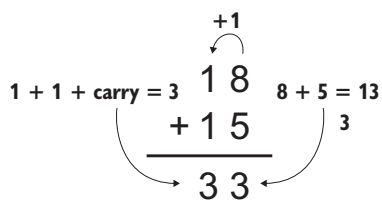


Figure 2.5 Illustrating the carry when summing $18 + 15$

The carry is such an important concept in this program that it deserves testing. This is what I meant in listing 2.9 when I said to pay extra attention to specific (business) rules and logic:

- *Carry*—Let's try sums that require carrying in many different ways. These are good places to start:
 - Sum without a carry
 - Sum with a carry: one carry at the beginning
 - Sum with a carry: one carry in the middle
 - Sum with a carry: many carries
 - Sum with a carry: many carries, not in a row
 - Sum with a carry: carry propagated to a new (most significant) digit

Domain knowledge is still fundamental to engineer good test cases

Up to this point, this chapter may have given you the impression that if you analyze every parameter of the method, you can derive all the test cases you need. Life would be much easier if that were true!

Analyzing parameters, even without much domain knowledge, will help you uncover many bugs. However, having a deep understanding of the requirements is still key in devising good test cases. In the current example, the requirements do not discuss the carry. We devised many tests around the carry because we have a deep knowledge of the problem. We build up knowledge over time; so although the systematic approaches I discuss will help you uncover many common bugs, it is your job to learn about the domain of the software system you're working on. (And if you wrote the code, you have an advantage: you know it deeply!)

The only boundary worth testing is the following: ensuring that cases such as $99 + 1$ (where the final number is carried to a new, most significant digit) are covered. This comes from the last partition derived when analyzing the *carry*: "Sum with a carry: carry propagated to a new (most significant) digit."

With all the inputs and outputs analyzed, it is time to derive concrete test cases. Let's apply the following strategy:

- 1 Test nulls and empties just once.
- 2 Test numbers with single digits just once.
- 3 Test numbers with multiple digits, with `left` and `right` having the same and different lengths. We will be thorough and have the same set of tests for both equal and different lengths, and we will duplicate the test suite to ensure that everything works if `left` is longer than `right` or vice versa.
- 4 We will exercise the zeroes on the left, but a few test cases are enough.
- 5 Test the boundary.

Let's look at the specific test cases:

- Nulls and empties
 - T1: left null
 - T2: left empty
 - T3: right null
 - T4: right empty
- Single digits
 - T5: single digit, no carry
 - T6: single digit, carry
- Multiple digits
 - T7: no carry
 - T8: carry in the least significant digit
 - T9: carry in the middle
 - T10: many carries
 - T11: many carries, not in a row
 - T12: carry propagated to a new (now most significant) digit
- Multiple digits with different lengths (one for left longer than right, and one for right longer than left)
 - T13: no carry
 - T14: carry in the least significant digit
 - T15: carry in the middle
 - T16: many carries
 - T17: many carries, not in a row
 - T18: carry propagated to a new (now most significant) digit
- Zeroes on the left
 - T19: no carry
 - T20: carry
- Boundaries
 - T21: carry to a new most significant digit, by one (such as $99 + 1$).

Now we transform them into automated test cases, as shown in listing 2.10. A few remarks about this listing:

- This test uses the `ParameterizedTest` feature from JUnit. The idea is that we write a single generic test method that works like a skeleton. Instead of having hard-coded values, it uses variables. The concrete values are passed to the test method later. The `testCases()` method provides inputs to the `shouldReturnCorrectResult` test method. The link between the test method and the method source is done through the `@MethodSource` annotation. JUnit offers other ways to provide inputs to methods, such as inline comma-separated values (see the `@CsvSource` annotation in the documentation).

- The `numbers()` helper method receives a list of integers and converts it to a `List<Integer>`, which the method under test receives. This helper method increases the legibility of the test methods. (For the Java experts, the `Arrays.asList()` native method would have yielded the same result.)

Listing 2.10 Tests for the add method

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.Arguments;
import org.junit.jupiter.params.provider.MethodSource;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.junit.jupiter.params.provider.Arguments.of;

public class NumberUtilsTest {

    @ParameterizedTest
    @MethodSource("testCases")
    void shouldReturnCorrectResult(List<Integer> left,
                                  List<Integer> right, List<Integer> expected) {
        → assertThat(new NumberUtils().add(left, right))
           .isEqualTo(expected);
    }

    static Stream<Arguments> testCases() {

        return Stream.of(
            of(null, numbers(7,2), null), // T1
            of(numbers(), numbers(7,2), numbers(7,2)), // T2
            of(numbers(9,8), null, null), // T3
            of(numbers(9,8), numbers(), numbers(9,8)), // T4

            of(numbers(1), numbers(2), numbers(3)), // T5
            of(numbers(9), numbers(2), numbers(1,1)), // T6

            of(numbers(2,2), numbers(3,3), numbers(5,5)), // T7
            of(numbers(2,9), numbers(2,3), numbers(5,2)), // T8
            of(numbers(2,9,3), numbers(1,8,3), numbers(4,7,6)), // T9
            of(numbers(1,7,9), numbers(2,6,8), numbers(4,4,7)), // T10
            of(numbers(1,9,1,7,1), numbers(1,8,1,6,1),
               numbers(3,7,3,3,2)), // T11
            of(numbers(9,9,8), numbers(1,7,2), numbers(1,1,7,0)), // T12

            of(numbers(2,2), numbers(3), numbers(2,5)), // T13.1
            of(numbers(3), numbers(2,2), numbers(2,5)), // T13.2
            of(numbers(2,2), numbers(9), numbers(3,1)), // T14.1
            of(numbers(9), numbers(2,2), numbers(3,1)), // T14.2
            of(numbers(1,7,3), numbers(9,2), numbers(2,6,5)), // T15.1
            of(numbers(9,2), numbers(1,7,3), numbers(2,6,5)), // T15.2
        );
    }
}
```

Annotations and Callouts:

- Calls the method under test, using the parameterized values**: Points to the `assertThat` call in the `shouldReturnCorrectResult` method.
- A parameterized test is a perfect fit for these kinds of tests!**: Points to the `@ParameterizedTest` annotation.
- Indicates the name of the method that will provide the inputs**: Points to the `@MethodSource("testCases")` annotation.
- One argument per test case**: Points to the `testCases` method.
- Tests with nulls and empties**: Points to test cases T1, T2, T3, and T4.
- Tests with single digits**: Points to test cases T5 and T6.
- Tests with multiple digits**: Points to test cases T7 through T12.
- Tests with multiple digits, different length, with and without carry (from both sides)**: Points to test cases T13.1 through T15.2.

```

Tests with multiple
digits, different
length, with and
without carry
(from both sides)
    of(numbers(3,1,7,9), numbers(2,6,8), numbers(3,4,4,7)), // T16.1
    of(numbers(2,6,8), numbers(3,1,7,9), numbers(3,4,4,7)), // T16.2
    of(numbers(1,9,1,7,1), numbers(2,1,8,1,6,1),
        numbers(2,3,7,3,3,2)), // T17.1
    of(numbers(2,1,8,1,6,1), numbers(1,9,1,7,1),
        numbers(2,3,7,3,3,2)), // T17.2
    of(numbers(9,9,8), numbers(9,1,7,2), numbers(1,0,1,7,0)), // T18.1
    of(numbers(9,1,7,2), numbers(9,9,8), numbers(1,0,1,7,0)), // T18.2

Tests with zeroes
on the left
    of(numbers(0,0,0,1,2), numbers(0,2,3), numbers(3,5)), // T19
    of(numbers(0,0,0,1,2), numbers(0,2,9), numbers(4,1)), // T20

    of(numbers(9,9), numbers(1), numbers(1,0,0)) // T21 ← The boundary
    };                                         test
}

private static List<Integer> numbers(int... nums) { ←
    List<Integer> list = new ArrayList<>();
    for(int n : nums)
        list.add(n);
    return list;
}
}

```

Auxiliary method that produces a list of integers. Auxiliary methods are common in test suites to help developers write more maintainable test code.

Interestingly, a lot of these test cases break! See the JUnit report in figure 2.6. For example, take the first failing test, T6 (single digit with a carry). Given `left = [9]` and `right = [2]`, we expect the output to be `[1,1]`. But the program outputs `[1]`! T12 (“carry propagated to a new (now most significant) digit”) also fails: given `left = [9,9,8]` and `right = [1,7,2]`, we expect the output to be `[1,1,7,0]`, but it is `[1,7,0]`. The program cannot handle the carry when the carry needs to become a new leftmost digit.

What a tricky bug! Did you see it when we wrote the method implementation?

There is a simple fix: all we need to do is add the carry at the end, if necessary. Here’s the implementation.

Listing 2.11 First bug fix in the add program

```

// ... all the code here ...
if (carry > 0)
    result.addFirst(carry);
return result;

```

With these tests passing, we see that the program does not handle zeroes to the left. When `left = [0,0,0,1,2]` and `right = [0,2,3]`, we expect the output to be `[3,5]`, but the program returns `[0,0,0,3,5]`. The fix is also straightforward: remove the zeroes on the left before returning the result (listing 2.12).

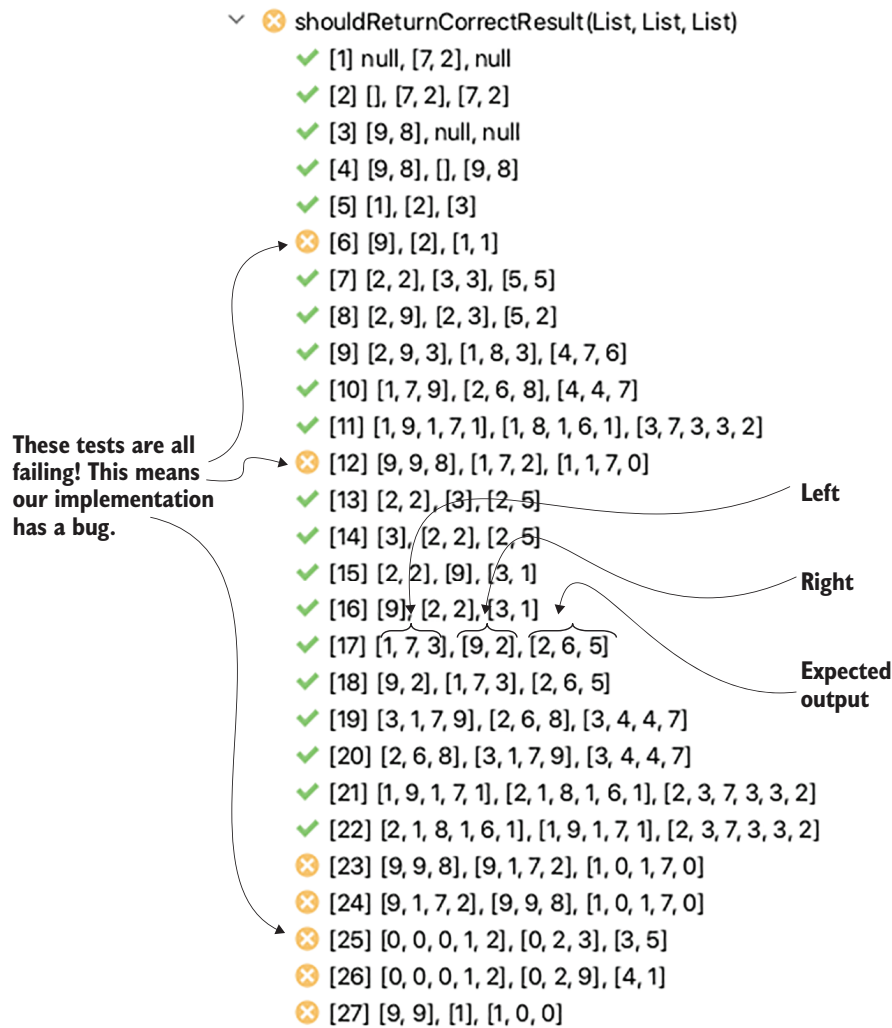


Figure 2.6 The results of the test cases we just created. A lot of them fail, indicating that the program has a bug!

Listing 2.12 Second bug fix in the add program

```
// ... previous code here...
```

```
if (carry > 0)
    result.addFirst(carry);
```

```
while (result.size() > 1 && result.get(0) == 0)
    result.remove(0);
```

```
return result;
```

Removes leading zeroes from the result

We're only missing test cases to ensure that the pre-condition holds that each digit is a number between 0 and 9. All we need to do is pass various invalid digits. Let's do it directly in the JUnit test as follows.

Listing 2.13 Tests for a pre-condition of the add program

```
@ParameterizedTest
@MethodSource("digitsOutOfRange")
void shouldThrowExceptionWhenDigitsAreOutOfRange(List<Integer> left,
    List<Integer> right) {
    assertThatThrownBy(() -> new NumberUtils().add(left, right))
        .assertInstanceOf(IllegalArgumentException.class);
}

static Stream<Arguments> digitsOutOfRange() {
    return Stream.of(
        of(numbers(1, -1, 1), numbers(1)),
        of(numbers(1), numbers(1, -1, 1)),
        of(numbers(1, 10, 1), numbers(1)),
        of(numbers(1), numbers(1, 11, 1))
    );
}
```

← A parameterized test also fits well here.

← Asserts that an exception happens

← Passes invalid arguments

All tests are now passing. Given the thoroughness of our test suite, I feel confident enough to move on.

NOTE Interestingly, the bugs we found in this example were caused not by buggy code but by a lack of code. This is a common type of bug, and it can be caught by specification testing. When in doubt, write a test! Writing automated (unit) test cases is so quick that they let you easily see what happens. Having too many useless tests is a problem, but a couple will not hurt.

2.4 *Specification-based testing in the real world*

Now that you have a clear understanding of how to systematically devise test cases based on specifications, here are a few pragmatic tips I have learned over the years.

2.4.1 *The process should be iterative, not sequential*

Describing iterative processes in writing is challenging. My explanation may have given you the impression that this process is fully sequential and that you move to the next step only when you have completed the previous one. However, the entire process is meant to be iterative. In practice, I go back and forth between the different steps. Often, when I'm writing test cases, I notice that I missed a partition or boundary, and I go back and improve my test suite.

2.4.2 How far should specification testing go?

The pragmatic answer to this question is to understand the risks of a failure. What would be the cost of a failure in that part of the program? If the cost is high, it may be wise to invest more in testing, explore more corner cases, and try different techniques to ensure quality. But if the cost is low, being less thorough may be good enough. Personally, I stop testing when I have been through all the steps a couple of times and cannot see a case I am not testing.

2.4.3 Partition or boundary? It does not matter!

When you are exploring inputs and outputs, identifying partitions, and devising test cases, you may end up considering a boundary to be an exclusive partition and not a boundary between two partitions. It does not matter if a specific case emerges when you are identifying partitions or in the boundaries step. Each developer may interpret the specification differently, and minor variations may result. The important thing is that the test case emerges and the bug will not slip into the program.

2.4.4 On and off points are enough, but feel free to add in and out points

On and off points belong to specific partitions, so they also serve as concrete test cases for the partitions. This means testing all the boundaries of your input domain is enough. Nevertheless, I often try some in and out points in my tests. They are redundant, because the on and off points exercise the same partition as the in and out points; but these extra points give me a better understanding of the program and may better represent real-life inputs. Striving for the leanest test suite is always a good idea, but a few extra points are fine.

2.4.5 Use variations of the same input to facilitate understanding

You can simplify your understanding of the different test cases by using the same input seed for all of them, as we noticed in an observational study with professional developers described in my paper with Treude and Zaidman (2021). For each partition, you then make small modifications to the input seed: just enough to meet the criteria of that partition. In the chapter example, all the test cases are based on the string “abc”; as soon as one test case fails, it is easy to compare it to similar inputs from other test cases that pass.

Note that this trick goes against the common testing idea of varying inputs as much as possible. Varying inputs is essential, as it allows us to explore the input space and identify corner cases. However, when doing specification-based testing, I prefer to focus on rigorously identifying and testing partitions. Later in the book, we will write test cases that explore the input domain in an automated fashion via property-based testing in chapter 5.

2.4.6 *When the number of combinations explodes, be pragmatic*

If we had combined all the partitions we derived from the `substringsBetween` program, we would have ended up with 320 tests. This number is even larger for more complex problems. Combinatorial testing is an entire area of research in software testing; I will not dive into the techniques that have been proposed for such situations, but I will provide you with two pragmatic suggestions.

First, reduce the number of combinations as much as possible. Testing exceptional behavior isolated from other behaviors (as we did in the example) is one way to do so. You may also be able to leverage your domain knowledge to further reduce the number of combinations.

Second, if you are facing many combinations at the method level, consider breaking the method in two. Two smaller methods have fewer things to test and, therefore, fewer combinations to test. Such a solution works well if you carefully craft the method contracts and the way they should pass information. You also reduce the chances of bugs when the two simple methods are combined into a larger, more complex one.

2.4.7 *When in doubt, go for the simplest input*

Picking concrete input for test cases is tricky. You want to choose a value that is realistic but, at the same time, simple enough to facilitate debugging if the test fails.

I recommend that you avoid choosing complex inputs unless you have a good reason to use them. Do not pick a large integer value if you can choose a small integer value. Do not pick a 100-character string if you can select a 5-character string. Simplicity matters.

2.4.8 *Pick reasonable values for inputs you do not care about*

Sometimes, your goal is to exercise a specific part of the functionality, and that part does not use one of the input values. You can pass any value to that “useless” input variable. In such scenarios, my recommendation is to pass realistic values for these inputs.

2.4.9 *Test for nulls and exceptional cases, but only when it makes sense*

Testing nulls and exceptional cases is always important because developers often forget to handle such cases in their code. But remember that you do not want to write tests that never catch a bug. Before writing such tests, you should understand the overall picture of the software system (and its architecture). The architecture may ensure that the pre-conditions of the method are satisfied before calling it.

If the piece of code you are testing is very close to the UI, exercise more corner cases such as null, empty strings, uncommon integer values, and so on. If the code is far from the UI and you are sure the data is sanitized before it reaches the component under test, you may be able to skip such tests. Context is king. Only write tests that will eventually catch a bug.

2.4.10 Go for parameterized tests when tests have the same skeleton

A little duplication is never a problem, but a lot of duplication is. We created 21 different tests for the `substringsBetween` program. The test code was lean because we grouped some of the test cases into single test methods. Imagine writing 21 almost-identical test cases. If each method took 5 lines of code, we would have a test class with 21 methods and 105 lines. This is much longer than the test suite with the parameterized test that we wrote.

Some developers argue that parameterized tests are confusing. Deciding whether to use JUnit test cases or parameterized tests is, most of all, a matter of taste. I use parameterized tests when the amount of duplication in my test suite is too large. In this chapter, I leaned more toward JUnit test cases: lots of test cases logically grouped in a small set of test methods. We discuss test code quality further in chapter 10.

2.4.11 Requirements can be of any granularity

The seven-step approach I propose in this chapter works for requirements of any granularity. Here, we applied it in a specification that could be implemented by a single method. However, nothing prevents you from using it with larger requirements that involve many classes. Traditionally, specification-based testing techniques focus on black-box testing: that is, testing an entire program or feature, rather than unit-testing specific components. I argue that these ideas also make sense at the unit level.

When we discuss larger tests (integration testing), we will also look at how to devise test cases for sets of classes or components. The approach is the same: reflect on the inputs and their expected outputs, divide the domain space, and create test cases. You can generalize the technique discussed here to tests at any level.

2.4.12 How does this work with classes and state?

The two methods we tested in this chapter have no state, so all we had to do was think of inputs and outputs. In object-oriented systems, classes have state. Imagine a `ShoppingCart` class and a behavior `totalPrice()` that requires some `CartItem`s to be inserted before the method can do its job. How do we apply specification-based testing in this case? See the following listing.

Listing 2.14 `ShoppingCart` and `CartItem` classes

```
public class ShoppingCart {

    private List<CartItem> items = new ArrayList<CartItem>();

    public void add(CartItem item) {
        this.items.add(item);
    }

    public double totalPrice() {
        double totalPrice = 0;
        for (CartItem item : items) {

```

← Adds items
to the cart

← Loops through all the items
and sums up the final price

```

        totalPrice += item.getUnitPrice() * item.getQuantity();
    }
    return totalPrice;
}
}

public class CartItem {

    private final String product;
    private final int quantity;
    private final double unitPrice;

    public CartItem(String product, int quantity,
        double unitPrice) {
        this.product = product;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
    }

    // getters
}

```

A simple class that
represents an item
in the cart

Nothing changes in the way we approach specification-based testing. The only difference is that when we reflect about the method under test, we must consider not only the possible input parameters, but also the state the class should be in. For this specific example, looking at the expected behavior of the `totalPrice` method, I can imagine tests exercising the behavior of the method when the cart has zero items, a single item, multiple items, and various quantities (plus corner cases such as nulls). All we do differently is to set up the class's state (by adding multiple items to the cart) before calling the method we want to test, as in the following listing.

Listing 2.15 Tests for the `ShoppingCart` class

```

import org.junit.jupiter.api.Test;
import static org.assertj.core.api.Assertions.assertThat;

public class ShoppingCartTest {

    private final ShoppingCart cart = new ShoppingCart();

    @Test
    void noItems() {
        assertThat(cart.totalPrice())
            .isEqualTo(0);
    }

    @Test
    void itemsInTheCart() {
        cart.add(new CartItem("TV", 1, 120));
        assertThat(cart.totalPrice())
            .isEqualTo(120);
    }
}

```

Having the cart as a
field means we don't
have to instantiate it
for every test. This is
a common technique
to improve legibility.

Asserts that
an empty cart
returns 0

Asserts that it
works for a single
item in the cart ...

```

    cart.add(new CartItem("Chocolate", 2, 2.5));
    assertThat(cart.totalPrice(),
        .isEqualTo(120 + 2.5*2);
    }
}

```

← ... as well as for many items in the cart.

Again, the mechanics are the same. We just have to take more into consideration when engineering the test cases.

2.4.13 The role of experience and creativity

If two testers performed the specification-based testing technique I described earlier in the same program, would they develop the same set of tests? Ideally, but possibly not. In the `substringsBetween()` example, I would expect most developers to come up with similar test cases. But it is not uncommon for developers to approach a problem from completely different yet correct angles.

I am trying to reduce the role of experience and creativity by giving developers a process that everybody can follow, but in practice, experience and creativity make a difference in testing. We observed that in a small controlled experiment (Yu, Treude, and Aniche, 2019).

In the `substringsBetween()` example, experienced testers may see more complicated test cases, but a novice tester may have difficulty spotting those. A more experienced tester may realize that spaces in the string play no role and skip this test, whereas a novice developer may be in doubt and write an extra “useless” test. This is why I like the specification-based testing systematic approach I described in this chapter: it will help you remember what to think about. But it is still up to you to do the thinking!

Exercises

- 2.1** Which statement is false about applying the specification-based testing method on the following Java method?

```

/**
 * Puts the supplied value into the Map,
 * mapped by the supplied key.
 * If the key is already in the map, its
 * value will be replaced by the new value.
 *
 * NOTE: Nulls are not accepted as keys;
 * a RuntimeException is thrown when key is null.
 *
 * @param key the key used to locate the value
 * @param value the value to be stored in the HashMap
 * @return the prior mapping of the key,
 * or null if there was none.
 */
public V put(K key, V value) {
    // implementation here
}

```

- A** The specification does not specify any details about the value input parameter, and thus, experience should be used to partition it (for example, value being null or not null).
- B** The number of tests generated by the category/partition method can grow quickly, as the chosen partitions for each category are later combined one by one. This is not a practical problem for the `put()` method because the number of categories and partitions is small.
- C** In an object-oriented language, in addition to using the method's input parameters to explore partitions, we should also consider the object's internal state (the class's attributes), as it can also affect the method's behavior.
- D** With the available information, it is not possible to perform the category/partition method, as the source code is required for the last step (adding constraints).

- 2.2** Consider a `find` program that finds occurrences of a pattern in a file. The program has the following syntax:

```
find <pattern> <file>
```

After reading the specification and following specification-based testing, a tester devised the following partitions:

- A** Pattern size: empty, single character, many characters, longer than any line in the file
- B** Quoting: pattern is quoted, pattern is not quoted, pattern is improperly quoted
- C** Filename: good filename, no filename with this name, omitted
- D** Occurrences in the file: none, exactly one, more than one
- E** Occurrences in a single line, assuming the line contains the pattern: one, more than one

Now the number of combinations is too high. What actions could we take to reduce the number of combinations?

- 2.3** Postal codes in some imaginary country are always composed of four numbers and two letters: for example, 2628CD. Numbers are in the range [1000, 4000]. Letters are in the range [C, M].

Consider a program that receives two inputs—an integer (for the four numbers) and a string (for the two letters)—and returns `true` (valid postal code) or `false` (invalid postal code). The boundaries for this program appear to be straightforward:

- A** Anything below 1000: invalid
- B** [1000, 4000]: valid
- C** Anything above 4000: invalid

- D [A, B]: invalid
- E [C, M]: valid
- F [N, Z]: invalid

Based on what you as a tester assume about the program, what other corner or boundary cases can you come up with? Describe these invalid cases and how they may exercise the program based on your assumptions.

- 2.4 A program called `FizzBuzz` does the following: given an integer `n`, return the string formed from the number followed by "!". If the number is divisible by 3, use "Fizz" instead of the number; and if the number is divisible by 5, use "Buzz" instead of the number, and if the number is divisible by both 3 and 5, use "Fizz-Buzz" instead of the number.

Examples:

- A The integer 3 yields "Fizz!"
- B The integer 4 yields "4!"
- C The integer 5 yields "Buzz!"
- D The integer 15 yields "FizzBuzz!"

A novice tester is trying to devise as many tests as possible for the `FizzBuzz` method and comes up with the following:

- A T1 = 15
- B T2 = 30
- C T3 = 8
- D T4 = 6
- E T5 = 25

Which of these tests can be removed while maintaining a good test suite? Which concept can we use to determine the test(s) that can be removed?

- 2.5 A game has the following condition: `numberOfPoints <= 570`. Perform boundary analysis on the condition. What are the on and off points?
- A On point = 570, off point = 571
 - B On point = 571, off point = 570
 - C On point = 570, off point = 569
 - D On point = 569, off point = 570
- 2.6 Perform boundary analysis on the following equality: `x == 10`. What are the on and off points?

Summary

- Requirements are the most important artifact we can use to generate tests.
- Specification-based testing techniques help us explore the requirements in a systematic way. For example, they help us examine the domain space of the different input variables and how they interact with each other.

- I propose a seven-step approach for specification testing: (1) understand the requirements, (2) explore the program if you do not know much about it, (3) judiciously analyze the properties of the inputs and outputs and identify the partitions, (4) analyze the boundaries, (5) devise concrete test cases, (6) implement the concrete test cases as automated (JUnit) tests, and (7) use creativity and experience to augment the test suite.
- Bugs love boundaries. However, identifying the boundaries may be the most challenging part of specification testing.
- The number of test cases may be too large, even in simpler programs. This means you must decide what should be tested and what should not be tested.